Feedback Report 01

Thanks again for your effort on the MVP plan and system flows. I can see the depth of thinking you've put into it—especially across the infra setup, data handling, and synthetic conversation generation. You've clearly got a strong foundation in building intelligent pipelines, and it's obvious you're comfortable thinking in full-stack systems.

That said, I want to be candid about where the plan **does** and **doesn't** align with AUO—and what we need from this MVP to make it truly valuable with real design partners.

✅ What You Did Well

1. Full-Stack Awareness

You mapped out a complete AWS-based AI stack with clear stages: ingest, convert, train, deploy, retrieve. That shows technical maturity and ownership.

2. Multi-Modal Input Handling

You scoped multiple input types (PDFs, specs, reviews, images), which reflects a real understanding of creative workflows where input formats vary widely.

3. Hybrid Memory Model

Your use of Neo4j alongside vector search is strong. Graph for structured past decisions + semantic RAG is a smart combo for creative recall.

> This all shows you're a systems builder—not just a tool tinkerer. That's a serious asset.

❌ Where the Gaps Are

1. You’re Optimizing for Completeness—Not Decision Impact

AUO’s goal isn’t technical elegance—it’s helping a team make a *better decision in the moment*. Your architecture treats that outcome as a hopeful result of the system, not the core product requirement.

> The test isn’t “Did it work?”—it’s “Did it help a decision stick?”

---

2. There’s No Agentic Behavior Modeled

You’ve built components, not collaborators. AUO’s architecture needs intelligent roles that can:

- Choose when to surface insight
- Prioritize relevant nudges
- Suppress unnecessary noise
- Defer when uncertainty is high

Right now, everything is passive and piped. There’s no agentic sequencing or timing. This is critical to getting trust and usability right.

---

3. You Missed the MVP Wedge

We're not validating an ecosystem. We're validating whether **10 specific nudges**, from pre-uploaded data, can:

- Be triggered by meeting speech
- Deliver in under 15 seconds
- Be interacted with (pin, ask, skip)
- Actually help the team align faster

None of that loop is implemented or modeled clearly in your current proposal.

---

4. You Didn’t Define the Nudges

This is the core product. Without clearly modeled nudges, we don’t know what AUO *does*:

- What exactly gets surfaced?
- What triggers it?
- What context is shown?
- What action can the user take?

This leaves a critical blind spot—one that no infra or pipeline can fill alone.

🧠 What This Signals

You think in infrastructure and execution. That's a real strength. But the AUO MVP isn't a backend problem—it's an **interaction loop challenge**. And it demands fluency in how decisions happen inside a meeting, in real time.

Right now, the system you've proposed feels reactive—not behaviorally intelligent.

🔁 The Path Forward

To build the MVP we need to test with real design partners, we need one working slice of AUO's behavior. Not a full system—just a focused loop that proves the value of live nudging.

AUO MVP Builder – Deployment-Ready Plan

🎯 **Goal:** Deliver a testable MVP that runs *live in meetings*, supports 10 defined nudge types, and pulls only from **pre-uploaded RAG sources**.

✅ SCOPE AT A GLANCE

Pillar	Requirement
Nudges	10 real-time nudge types (e.g. constraints, feedback, visual prompts)
Data Prep	Uploads (PDF, notes, specs) pre-meeting—parsed and embedded
RAG	Retrieval must pull from that context only, with traceability
Latency	Nudges should surface within 15 seconds

Pillar	Requirement
Usage Format	Textual insight + optional image; user can pin or ask about it
Delivery Ready	Usable with real design teams within 2–3 weeks

🧩 TECHNICAL SYSTEM DESIGN (Required Features)

Component	Notes
Pre-Meeting Data Ingest	Upload interface → parse → embed
RAG Setup	Retrieve clean, explainable snippets with source
Nudge Engine	Logic or template system for 10 nudge types
Trigger Buffer	Catch decision patterns from transcript windows
Latency Benchmarking	Track speech → nudge timing
Insight Object Format	`trigger`, `insight`, `source`, `confidence`, `action`
Memory Layer (Optional)	Link to prior insights or meetings (Neo4j or JSON)
UI/Export	Webhook, CLI, or UI endpoint for pinning and review

🎯 10 NUDGE TYPES TO BE SUPPORTED

Type	Example
Constraint Reminder	“This SKU must stay under $18.42 landed.”
Feedback Stat	“32% of reviews cite tight leg fit.”
Past Decision Recall	“You dropped this neckline in March due to bulk.”
Visual Suggestion	“Here’s a less-boxy variant.”
Timeline Risk	“This path adds 3 weeks to ship date.”
Competitive Flag	“Nike just launched a similar mesh cut.”
Stakeholder Comment	“Finance flagged this in last sync.”
Design Intent Reminder	“Brief emphasized ‘technical minimalism.’”
Conflict Recall	“Same concern was raised 2 weeks ago.”
Price Warning	“This configuration puts you $4 over margin.”

🛠️ DELIVERY CHECKLIST

- [ ] RAG pipeline working end-to-end
- [ ] 10 nudge types modeled and functioning
- [ ] Listener + trigger logic operational
- [ ] Basic interaction surface (pin, act, skip)
- [ ] Example data package to test
- [ ] Test live with 2–3 partner meetings

🧠 CRITICAL THINKING REQUIRED

- How will insights be ranked or suppressed?
- What signals “too soon” or “not relevant”?
- What fallback happens when RAG fails?
- How do we keep nudges *explainable and trustable*?